Speaker Voice Similarity Analysis and Evaluation

ARI5121 - APPLIED NLP ASSIGNMENT 2026

DR ANDREA DEMARCO

Introduction

In this assignment, you will be required to analyze and evaluate the prowess of a major pre-trained speech model for speaker voice similarity. This is a fundamental step prior to selecting a tool for use in a more fully-fledged Speaker Identification (SID) or Speaker Diarization systems. In this assignment you will be provided with speech data for a number of speakers (285 in total) to perform the analysis.

Dataset

The dataset you will work with is the Accents of the British Isles (ABI-1) Corpus. The corpus is split into 14 folders, each folder containing data from a single accent, as follows: BRM (Birmingham), CRN (Cornwall), EAN (East Anglia), EYK (East Yorkshire), GLA (Glasgow), ILO (Inner London), LAN (Lancashire), LVP (Liverpool), NCL (Newcastle), NWA (Northern Wales), ROI (Republic of Ireland), SHL (Scottish Highlands), SSE (Standard Southern English), ULS (Ulster).

Furthermore, within each accent folder, is data from approximately 20 speakers in separate male and female folders. Each speaker folder name serves as a unique ID for that speaker. For each speaker, you will find a number of .WAV files, coupled with transcriptions in .TRS or .TXT files. You will not need to make use of transcriptions for this assignment. You will also only need to utilise the .WAV files denoted by "shortpassage" in their filename. These .WAV files are of considerably long duration (around 40-60 seconds each), with the speaker reading a passage. **Hint:** Feel free to clean up the files, leaving only the required .wav files needed. You may do this manually or in Python.

Pre-Trained Model & Dataset

You shall use a Microsoft model (WavLM), which is an enhanced version of Wav2Vec2, and fine-tuned for speaker verification tasks. This model can be found on HuggingFace: Model (microsoft/wavlm-base-plus-sv). It is suggested to use the Python Transformers package to interact directly with HuggingFace. In your report, you will need to describe the model in brief.

Dataset can be downloaded from: ABI-1 Corpus

Similarity Function

The similarity function between two embeddings extracted from a model shall be the cosine similarity function. Implementations exist in most scientific or deep learning libraries. Essentially this metric is the cosine of the angle between two vectors. The similarity will be in the interval [-1, 1], -1 being orthogonally not similar, and +1 being exactly similar, with values in between.

Implementation Overview

1. Write code in Python, using any deep learning library of your choice.
2. Your core code should load the pre-trained model, transform two voice samples to embeddings using the model, and perform cosine similarity across these two voice sample embeddings. Any processing on the wav file, or on the embeddings (one per speech frame) that is required is up to you based on your research into how the model works.
3. Evaluation: Cross compare the embeddings of all speakers i.e. each sample from a speaker needs to be compared to all samples of the same speaker, and all samples of all other speakers. Based on the numerical scores you obtain, you can decide on a threshold for similarity which identifies a speaker correctly, whilst lowering misclassifications (above a certain threshold -> same speaker, below -> different speaker). Report on the system accuracy based on the threshold you choose.

Detailed Evaluation

Once the core implementation above is finished, proceed to perform a more detailed analysis as follows - this will be reflected in your report.

QUANTITATIVE EVALUATION METRICS

Accuracy

1. Measure the overall identification accuracy by computing how often the system correctly matches a speaker to their corresponding ID.
2. Divide the correctly identified speaker matches by the total number of trials.

Cosine Similarity Thresholds

1. As above, decide on your threshold for cosine similarity based on a few trials, to determine when two embeddings should be considered the same speaker.
2. Experiment with different thresholds and observe how this affects performance (e.g. increasing true positives but also false positives.

Confusion Matrix

1. Compute a confusion matrix to visualise the system's performance in classifying speakers correctly.
2. This can highlight which speakers are being confused with each other, providing insights into potential pitfalls with the dataset or model.

CLUSTERING ANALYSIS

1. Visually assess the embedding space by performing t-SNE or PCA dimensionality reduction on the embeddings of around 20 speakers, and plotting the result. Analyse if clusters correspond well with different speakers.

Report Structure

A. Introduction (10%)

- Briefly introduce the concept of speaker identification and how this can be constructed as a downstream task with a pre-trained speech model such as Wav2Vec2.
- State the objective of the experiment: to analyse and evaluate the performance of your chosen system on the provided dataset.

B. Data Preprocessing (10%)

- Describe the dataset, including the number of speakers, duration, and conditions (e.g., clean, noisy).
- Explain the preprocessing steps taken (resampling, silence removal, normalisation, etc.), if any.

C. Methodology (20%)

- Detail how embeddings were extracted using the chosen model.
- Explain how the embeddings were aggregated (e.g., averaging, pooling) and how similarity scores were calculated.
- Discuss any dimensionality reduction techniques applied.
- Provide details on the experimental setup and the evaluation pipeline.

D. Evaluation and Results (25%)

- Present the evaluation metrics (accuracy, confusion matrix, etc.) for the model.
- Show any visualization (e.g., t-SNE plots of embedding spaces) used to inspect the speaker separability.
- Detail the performance of the model quantitatively and qualitatively.

E. Analysis and Discussion (15%)

- Critically analyze the results. How did the model perform? Why do you think this was the case?
- Reflect on any limitations in the models and potential biases.

F. Conclusion and Future Work (5%)
- Summarize the key takeaways from the experiment.

G. Generative AI - Appendix (15%)
- You are to use Generative AI as an aid in solving the problem. Your interactions/chats with Generative AI should be given in full as an appendix, in chronological order. Further notes on the use of Generative AI and how this can be presented in your work can be found in the next section.

Use of Generative AI

The use of Generative AI tools to prepare any aspect of your work, should be in line with the following guidelines:

1. You are permitted to use Generative AI tools for this work, as long as you do not submit the actual AI-generated output as your own work for assessment.
2. Generative AI tools can be used to correct syntax and language, improving clarity of your work. However, if used it must be clearly noted.
3. It is important to understand that output provided by Generative AI may not always be reliable or up to date.
4. UM policy can be accessed here.

Furthermore:
1. Generating Original Content: using AI to write large sections or the entirety of the text is not acceptable, as this undermines academic integrity and originality.
2. Plagiarism: copying text generated by AI without proper attribution can result in unintentional plagiarism. Students are ultimately responsible for submitted work.
3. References: AI tools must be referenced including the name of the tool, as well as the access date and version of the tool being used e.g. GPT3 or GPT4.5.
4. Description of Use: Students must submit an appendix which includes the Generative AI prompts and output obtained. Furthermore, reports in general should aim to show how the student adapted the output for their own work.
5. Code Optimization: Students can use Generative AI tools for code debugging, architecture adjustments, programming style correction, function creation, code design changes.
6. Problem solving: Students can use Generative AI to seek inspiration for a solution to a given task/problem, stimulating creativity and fostering a structured approach to problem solving. Complete scripts generated by Generative AI are, however, not acceptable for submission. If there is reasonable suspicion that this has occurred, the coordinator reserves the right to call up the student for an interview in order to clarify the matter.

Deadline

The deadline for submission is 22nd May 2026.